GENESIS 2.5: CUDA and OpenCL acceleration for existing models of the GENESIS compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a]*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b]*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator. GENESIS 2.5 adds optional CUDA and OpenCL backends to its `hsolve` solver, including a GPU Hines elimination for dendritic-tree populations, without changes to existing models or scripts. On 160 000 compartments over 10 000 steps an A100 runs 24.0× faster end to end than the CPU solver. In double precision it matches the CPU solver to ten significant figures. Spiking networks with zero axonal delay run slower, because the host delivers spikes every step. The release also makes model construction linear in the common case and fixes two Hines-solver defects of GENESIS 2.4.

Keywords: GENESIS, PGENESIS, compartmental neural simulation, Hines solver, GPU, CUDA, OpenCL

Required Metadata

Current code version

Main text

1. Motivation and significance
   GENESIS [1] and its MPI version PGENESIS are among the longest-maintained compartmental simulators. Models are written as SLI scripts, cell files and channel prototypes, often accumulated over many

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.6.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/WarsawIQ/genesis-2.5` (release v2.6.0, archived at `https://doi.org/10.5281/zenodo.23298087`)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions); see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, CUDA C++, OpenCL C, MPI (PGENESIS), Python and POSIX shell (reproduction and analysis)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, flex, bison, ncurses. Optional: CUDA 12.x (tested 12.8 on NVIDIA A40 and A100), an OpenCL 1.2 runtime (tested NVIDIA, AMD ROCm 6.3.1, Mesa rusticl 26.1), MPICH or Open MPI for PGENESIS
C7	If available Link to developer documentation/manual	`README.md`, `reproduce/README.md`, `paper/docs/REPLICATION.md`
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

years, and the compute engine is `hsolve`, an implicit Hines solver [2] that compiles a model into a flat program at setup. Until this release every step of that program ran on the CPU.

Relationship to GENESIS 2.4. GENESIS 2.5 is an independently maintained derivative of GENESIS 2.4, released under the same licence as a fork of the project's repository (`genesis-sim/genesis-2.4`, last changed in 2019). It is not an official release of the GENESIS project. Its number continues GENESIS 2. GENESIS 3 took a different path, a

modularisation of the simulator into stand-alone components based on the Neurospaces project [3, 4, 5, 6, 7]; GENESIS 2.5 keeps the code base of GENESIS 2.4 and runs its scripts unchanged. This paper describes release v2.6.0 of GENESIS 2.5. The second author has used GENESIS and PGENESIS since his doctoral work [8] and met its maintainer, Dave Beeman, at the EU Advanced Course in Computational Neuroscience in Óbidos in 2001; the first author completed his PhD under the second author's supervision in 2023. On 30 September 2026 we offered the work to the original project: the fixes as a pull request, and the whole release as its next version. Dave Beeman had not replied by the time we submitted this revision; the solver and construction fixes are open as pull request #7 in that repository.

Alternatives. NEURON [9] with its engine CoreNEURON [10] and Arbor [11] run compartmental models on GPUs and are strong choices for a new model. For an existing GENESIS model each means re-expressing it in another description and showing that the port computes the same thing, which is itself experimental work. For the published network of Section 3 it took a transcription of the channels and a comparison of spike trains. NEST [12] and Brian 2 [13] simulate networks of point neurons, and GeNN [14] generates GPU code for such networks. GENESIS 2.5 runs existing GENESIS scripts on a GPU without porting them.

Contributions.

(a) CUDA and OpenCL backends for `hsolve`, selected at run time, in single or double precision, falling back to the CPU for anything they cannot compute.

(b) `hines_tree_eliminate`, a GPU Hines elimination for populations of dendritic trees.

(c) Two fixes to the Hines solver of GENESIS 2.4, which with several cells under one solver computed wrong voltages (backward Euler) or diverged and crashed (Crank–Nicolson), and model construction in linear time; both apply with or without a GPU and are open as a pull request to the original project.

(d) A comparison with NEURON, CoreNEURON and Arbor in which every arm is measured in one session, and a reproduction package that regenerates every number in this paper from raw data (Section 3).

2. Software description

(a) Software architecture

The backends sit behind one entry point in `hsolve` (Fig. 1). The SLI interpreter, the element tree, the scheduler and the PGE-

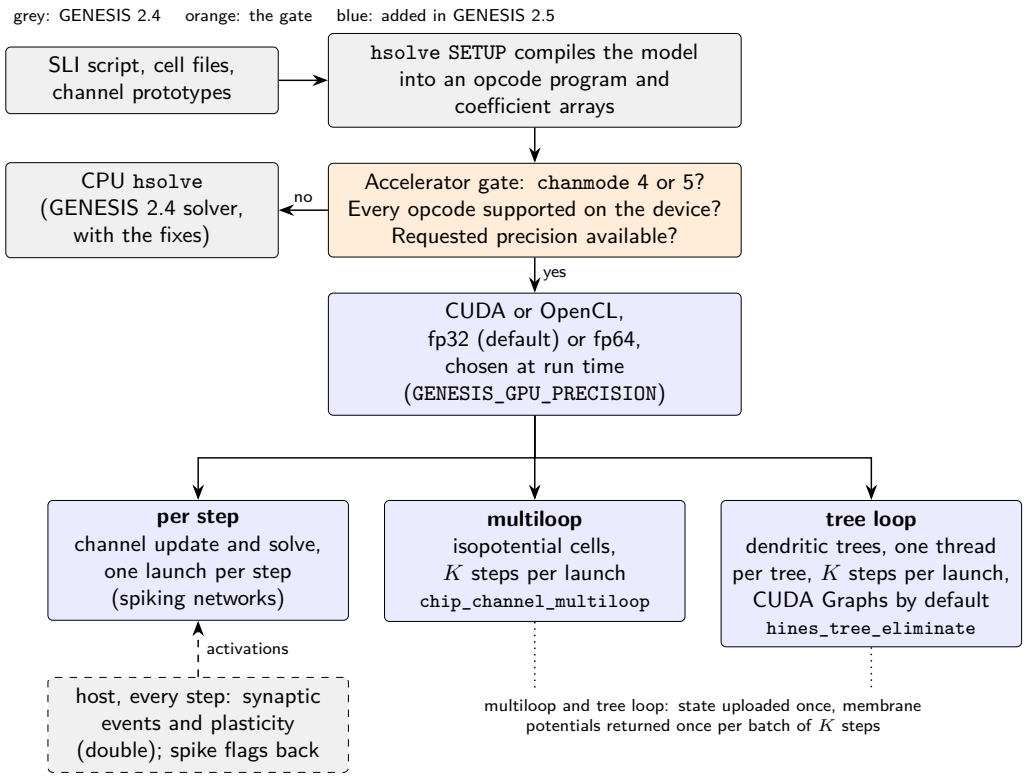


Figure 1: How a model reaches the GPU. `hsolve` compiles the model into an opcode program at `SETUP`. A gate checks the program against what the kernels implement; anything else stays on the CPU. Accepted models run as one of three dispatches: per step, batched over many steps for isopotential cells (`chip_channel_multiloop`), or batched for dendritic trees (`hines_tree_eliminate`). Synaptic events are delivered by the host every step.

NESIS MPI layer are those of GENESIS 2.4. A user selects a backend with `chanmode` and environment variables. Nothing in a model script changes.

An accepted model runs on one of three paths. *Per step*: the device updates channels and solves each step, and the host delivers synaptic events and spikes between steps. Networks whose cells exchange spikes take this path; on it a spike reaches some of its targets one step later than on the CPU (Section 3(d)). *Multiloop*: for isopotential cells without synaptic input, one launch runs K steps. *Tree loop*: for populations of dendritic trees, the channel update and `hines_tree_eliminate` run back to back for K steps. On the two batched paths the state goes to the device once and the membrane potentials come back once per batch, so a run of K steps costs one transfer instead of K.

Design decisions.

i. *The kernels interpret the solver's own program.* Both backends execute the opcode program `hsolve` already builds, so every model it accepts runs unchanged. GPU-native simulators generate code per mechanism instead. Interpreting costs divergence and a fixed setup time (Section 3); in exchange, existing models need no porting.

ii. *The tree solve needs no synchronisation.* The elimination program of a population is block-diagonal: each tree touches only its own rows. Each thread therefore runs the CPU's elimination for one tree, with no barrier. The only change needed was to split, per tree, the opcode that joins two trees in the CPU program.

iii. *What the kernels cannot compute is refused, not approximated.* Release v2.5, which the first version of this paper described, accepted models with synaptic, spike, GHK and concentration elements and returned plausible but wrong voltages. Such programs are now detected at `SETUP` and computed on the CPU.

iv. *The device owns the state it computes.* With synaptic input the host writes only the slots it owns each step. Release v2.5 uploaded the whole state, which overwrote gating variables the kernel had just updated.

Execution model. Four properties of the device code determine when the GPU pays off. *Work imbalance.* `hines_tree_eliminate` runs one thread per tree, so the threads of a warp wait for the largest tree among them. A population that mixes trees of 8 and 64 compartments in every warp takes 1.08× as long per step as uniform trees with the same total number of compartments. The CPU pays about the same for this mix (1.09×), so the waiting costs little. Grouping the same trees by size is slower (1.27×, E7), because it puts all the long trees on a few multiprocessors, which then finish last. *Divergence.* Compartments with different channel sets follow different opcode streams through the interpreter, so heterogeneous cells cost more than their compartment count suggests. *Transfers.* On the batched paths transfers are fixed per batch, not per step (above). On the per-step path they are paid every step. *Launch overhead.* In the tree loop on the A100 the host needs 6.2 µs per step to launch the two kernels, which run for 76 µs (Nsight Systems, $N = 10\,000$ trees). The launches overlap the kernels and do not limit the loop. CUDA Graphs, which re-

play a recorded sequence of launches, cut the host's part to 0.5 µs per step. In a run of $K = 50\,000$ steps they save $3.0 \pm 1.0\%$ of the run time on the A100 and $1.9 \pm 0.5\%$ on the A40 (E8). At $K = 5000$ they save $4.8 \pm 2.3\%$ on the A100 and $0.7 \pm 1.2\%$ on the A40, which is within the noise. Graphs change no result, so they are on by default for the tree loop. On the spiking network they make no difference within error ($-2.4 \pm 3.9\%$). There the cost is the host's event delivery and the synchronisation it forces every step. Moving spike delivery to the device is the next step for that workload.

(b) Software functionalities

Precision. Single precision is the default. `GENESIS_GPU_PRECISION=fp64` selects double precision at run time in the same binary, and a device without it refuses with a message and computes on the CPU. In fp64 the GPU reproduces the CPU solver to every printed digit (ten significant figures) on our verification models; in fp32 it agrees to within 2×10^{-7} V over one action potential on every device we tested. Synaptic weights and plasticity are computed by the synapse objects on the host in double precision in every mode. Only the resulting activation reaches the device. Section 3 measures the cost of double precision on each card and compares the two precisions over a 10 s run.

Portability. The OpenCL backend runs the same kernels on any OpenCL 1.2 device. We test it on the NVIDIA A40 and A100 and on the AMD Radeon 890M integrated GPU under two independent implementations, AMD ROCm and Mesa rusticl (E9). On the same chip the two differ: ROCm exposes double precision and rusticl does not, so under rusticl a request for fp64 is refused and the model computes on the CPU. On an integrated GPU that also drives the display, a long tree dispatch trips the driver's watchdog. There the backend declines the batched tree dispatch above 20 000 compartments, prints a message and runs the model step by step; `GENESIS_OCL_TREE_MAX_NCOMPTS` overrides the limit, and E9 checks that the limit is enforced. Under rusticl, which submits to the graphics queue, the same kind of limit stops the longest single-compartment run: the driver resets the GPU during the network of $N = 50\,000$ cells batched over 50 000 steps. Run one dispatch per step, the model takes 47.5 s, slower than the laptop's CPU at 32.2 s, while ROCm runs it batched in 5.7 s. Release v2.6.1 splits such a run into dispatches of about half a second, with the same

result, so under rusticl it too runs batched on the GPU.

(c) Sample code

A model needs no change as long as its solver uses `chanmode 4` or `chanmode 5`, modes GENESIS 2.4 already has. An existing script that builds its solver as

```
create hsolve /net/solver
setfield /net/solver path "/net/##[][TYPE=compartment]" \
    chanmode 4
call /net/solver SETUP
```

runs on the GPU when started with `nxgenesis` (CUDA) or `nxgenesis_ocl` (OpenCL) from the release. Batching and precision are chosen when the run starts, for example

```
GENESIS_CUDA_MULTILOOP=<K> GENESIS_GPU_PRECISION=fp64 \
    nxgenesis -batch model.g
```

The start-up lines name the backend, the precision and the dispatch path in use, and a model the device cannot compute is reported and run on the CPU.

3. Illustrative examples

Every number in this section was measured on release v2.6.0 on the UMCS "Lunar" cluster (inf02: Xeon Gold 6342 and NVIDIA A40; inf03: Xeon Platinum 8358 and NVIDIA A100). CPU arms are single-threaded. Times are wall-clock times of the whole process, and each value is a mean over replicates run in interleaved order. All runs, CPU arms included, were bound to the GPU's NUMA node, so both arms of a speedup used the same socket. A speedup is the ratio of two means; its uncertainty combines the two arms' relative sample standard deviations in quadrature. On inf02 the A40's socket is the slower one: a CPU arm takes 1.19× as long there as on the other socket, so against a CPU arm on the other socket the A40's speedups would be that much smaller. On inf03 the GPU's socket is slightly faster (0.99×). The CPU arms use the same `hsolve` mode as the GPU arms (`chanmode 4`). On the trees `chanmode 3` is faster still by 1.08×, so against it the tree speedups would be smaller by that factor, while `chanmode 1`, our CPU reference before this revision, is 4.2× slower with the same voltages.

(a) Speed on populations of cells

Table 2 gives the speedups for the single-compartment network, and Fig. 2 those for the dendritic trees, end to end and for the step phase alone. The end-to-end gain grows with run length as the fixed setup cost is amortised (Fig. 3). On the longest run of Fig. 3 the speedup is 17.1× on the A40 and 24.0× on the A100.

	NVIDIA A40		NVIDIA A100	
N	CUDA	OpenCL	CUDA	OpenCL
500	1.51 ± 0.03	1.49 ± 0.02	1.62 ± 0.05	1.77 ± 0.09
5 000	8.29 ± 0.08	8.31 ± 0.10	8.65 ± 0.31	9.16 ± 0.49
50 000	19.28 ± 0.20	19.33 ± 0.21	20.60 ± 0.24	20.64 ± 0.17

Table 2: Single-compartment network, $K = 50\,000$ steps: speedup of each backend over the CPU solver (E3, 10 replicates).

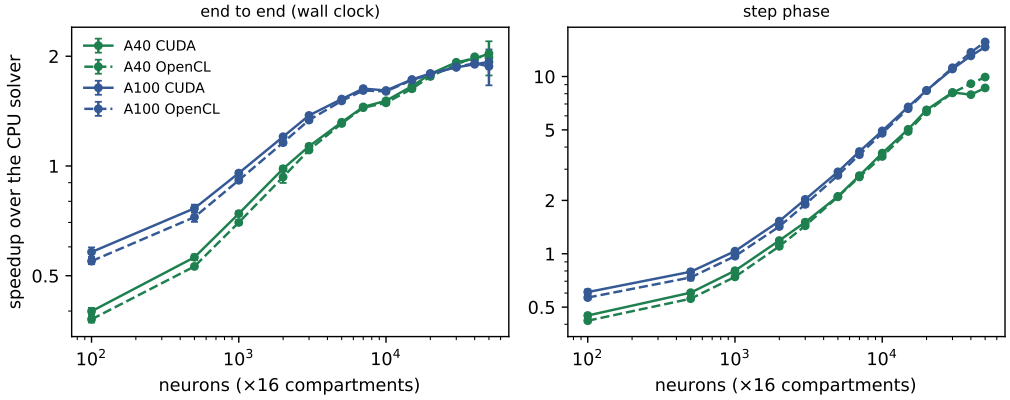


Figure 2: Dendritic trees, $N \times 16$ compartments, 200 steps: speedup over the CPU solver, end to end and step phase, on the A40 and A100 (E3).

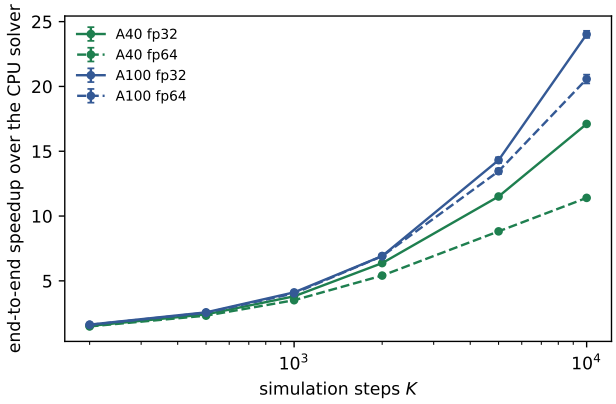


Figure 3: End-to-end speedup against run length, $10\,000 \times 16$ compartments, fp32 and fp64, both cards (E2).

Double precision takes 1.50× the fp32 time on the A40 and 1.17× on the A100.

(b) Against other simulators

Table 3 lists every arm of the spiking-network comparison, measured in one session on one node in interleaved order, so no ratio in it mixes machines or days. Arbor cannot express the benchmark's zero axonal delay, so its arm uses a delay of one step (50 µs). Arbor advances in epochs of half the shortest connection delay and exchanges spikes once per epoch [15]; with this delay it does so at least once per step, the same per-step exchange that keeps GENESIS from gaining on the GPU. We did not profile Arbor to confirm that this is where its time goes.

On the dendritic trees GENESIS pays more than Arbor at setup and less per step. In single precision it is slower than Arbor on short runs and faster beyond 48 ms of simulated time on the A100 and 13 ms on the A40; at the longest run it takes 0.74× Arbor's time on the A100 and 0.55× on the A40 (Fig. 4). In double precision, which Arbor always uses, GENESIS is slower at every run length we measured, taking 1.10× Arbor's time on the A100 and 1.05× on the A40 at the longest run.

We profiled GENESIS and CoreNEURON on one CPU core (E6). GENESIS is faster on both workloads, 2.22× on the network and 2.56× on the trees. GENESIS reads its channel gates from tables; CoreNEURON evaluates exponentials, and the exponential function alone takes 47% of its time on the network. On the network GENESIS spends 38% of its time on synapses, spike events and the calls that reach them, against CoreNEURON's 12%. This host-side work runs on the CPU every step, so the GPU cannot speed it up. On the trees the channels take 61% of GENESIS's time and the linear solve 21%, against 43% and 17% of CoreNEURON's. Most of the rest is model construction in both, plus idle library threads in CoreNEURON.

(c) When to use the GPU

Table 4 gathers what the measurements above say about where the accelerator pays off and where it does not.

(d) Correctness over a long run

The GPU and CPU solvers do not order spikes the same way. On the GPU a spike emitted in one step reaches its targets in the next; the CPU solver delivers it within the step to the cells after the sender in its loop. With this network's zero axonal delay the two schedules differ by one step (50 µs) for some synapses, and the network is chaotic, so it amplifies the difference. Over 10 s of the spiking network, twice the length of any comparison above, the GPU's spike train first departs from the CPU's after 2.65 ms,

Simulator	Hardware	Wall time (s)	Relative
GENESIS 2.5, one solver per cell (as published)	CPU	46.9 ± 1.0	1.36
GENESIS 2.5, one solver per layer	CPU	34.6 ± 0.9	1
GENESIS 2.5, one solver per layer, fp32	A100	39.6 ± 0.6	1.14
GENESIS 2.5, one solver per layer, fp64	A100	40.3 ± 0.5	1.17
NEURON 9.0.2, NMODL channels	CPU	100.1 ± 0.6	2.90
NEURON 9.0.2, ChannelBuilder channels	CPU	128.4 ± 1.5	3.71
CoreNEURON 9.0.2	CPU	76.5 ± 0.2	2.21
CoreNEURON 9.0.2	A100	27.1 ± 0.1	0.79
Arbor 0.10.0	A100	150.0 ± 1.8	4.34

Table 3: Vogels–Abbott COBAHH network [16, 17], 4000 cells, 5 s: wall time of every arm in one session on inf03 (E1, 5 replicates, interleaved). Relative: wall time over that of GENESIS with one solver per layer on the CPU; below 1 is faster.

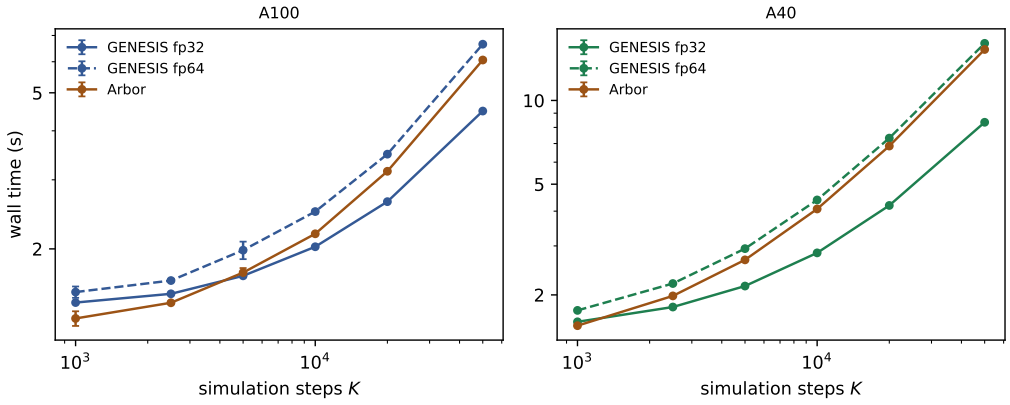


Figure 4: GENESIS and Arbor on the same GPU against run length, GENESIS in fp32 and fp64 (E4).

in fp64 and in fp32 alike. The two precisions part at the same step, so the cause is the order, not rounding. The CPU is just as sensitive to its own order: with one solver per cell instead of one per layer, the same network and seed depart from the first CPU run after 2.35 ms. From then on only statistics can be compared. Against the CPU's 27.2 Hz, the spike count differs by +0.9% in

Workload	GPU?	Why
One detailed morphology, or fewer than about 3000 trees	No	too few trees to fill the device; below the threshold the GPU is slower (Fig. 2)
Population of dendritic trees	Yes, from about 3000 cells (200 steps)	many trees fill the device (Fig. 2)
Isopotential cells without synapses	Yes	21× at $N = 50\,000$ (Table 2)
Spiking network, zero axonal delay	No: 1.14× slower	spikes are exchanged every step, so every step is a dispatch (Table 3)
Long runs	Gain grows with length	the fixed setup cost is amortised (Fig. 3)

Table 4: Where acceleration helps.

fp64, -0.3% in fp32 and -0.7% for the CPU in the other order; the KS distances between the interspike-interval distributions are 0.002, 0.003 and 0.006. Four other seeds, other draws of the same network, fire at 24.6–29.2 Hz with KS distances of 0.021–0.049. Each run repeats spike for spike on its own backend. So the GPU's spike count differs from the CPU's about as much as the CPU's own count changes under another update order. Its interspike intervals are closer to the CPU's than that run's are, and both differences are far smaller than the change from one draw of the network to another.

(e) Reproducing these results

Each number in this paper has a row in `reproduce/claims.csv` that names its data file, the rows and formula it comes from, and the script that measures it; a number taken from the literature names its source. `make_numbers.py` computes the values in the text from those files, and a lint lists any number typed by hand. `sh reproduce/run_all.sh` measures them again on the reader's hardware (about 15 minutes for the central claims, 95 for the sweeps; NVIDIA GPU, CUDA 12.x). `--cpu-only` reproduces the hardware-independent results in about two minutes on any Linux machine, and the same path runs in a container (`docker build -t genesis25-cpu . && docker run --rm genesis25-cpu`) with nothing else installed, as in our earlier pipeline for GENESIS

experiments [18]. Both run in the repository's CI on every commit. A generated table in the README says which results need which hardware. The comparison simulators and the reference model are built by committed recipes with pinned versions and checksums. Rebuilding them from an empty directory reproduced the published CoreNEURON GPU run to the spike.

4. Impact
   Two of the release's changes help every GENESIS user, with or without a GPU. The first is the solver fixes. In GENESIS 2.4, several cells under one `hsolve` silently gave wrong voltages under backward Euler: the first cell's were perturbed and the others were never computed. Under Crank–Nicolson the solve diverged or crashed. Networks are often built this way, so published results may be affected. GENESIS 2.5 computes each cell exactly as if it were alone, which a committed test checks against a single-cell reference. The fixes and their tests are open as a pull request to the original project (Section 1).
   The second is construction. Building a model took time quadratic in its size, growing as compartments$^{2.18}$: 272 000 compartments took 8 minutes to build, and at that rate the 1.7-million-compartment population of Fig. 5 would take about 7 hours. Four scans of the element tree are now constant- or linear-time. Construction grows as compartments$^{1.02}$, and the same population builds in 94.9 s. This holds for models built element by element, the common case; copying a whole prototype cell thousands of times under one parent is still quadratic. That population has 100 000 cells of 17 compartments, about three times the 31 000 neurons of a reconstructed neocortical microcircuit [19], though with far simpler cells. With the tree kernel, population studies such as parameter sweeps over thousands of morphological neurons can run on a workstation instead of in a cluster queue, in the simulator the models were written for. Table 4 lists the practical regimes, including where the GPU does not help.
   Two parts of this work apply beyond GENESIS. The first is the refusal rule: a backend that cannot compute a model should say so and leave it to the CPU. Release v2.5 approximated such models and returned plausible but wrong voltages. The second is the reproduction mechanism, a map from every published number to its raw data that regenerates the text and runs in CI. It does not depend on GENESIS and would serve any computational paper.
5. Conclusions
   GENESIS 2.5 runs existing GENESIS models on a GPU with no change to their SLI scripts, cell files or MPI workflows. The CUDA and

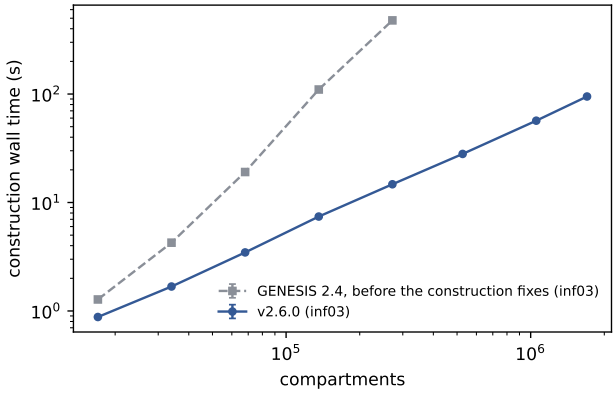


Figure 5: Model construction time against model size on inf03, 3 replicates each: GENESIS 2.4 before the construction fixes (the upstream branch without them, built on the node and timed the same way) and v2.6.0 (E3c). The code from before the fixes was measured up to 16 000 cells, where one build already takes 8 minutes.

OpenCL kernels execute `hsolve`'s own compiled program, in single or double precision chosen at run time. Populations of dendritic trees gain the most, up to 24.0× end to end. Single detailed morphologies and zero-delay spiking networks run slower on the GPU: the first has too few trees to fill the device, and the second needs host-side event delivery every step. For spiking networks the next step is spike delivery on the device. The solver and construction fixes help every GENESIS user and are open as a pull request to the original project. Every measured number in this paper can be regenerated from raw data in the repository, the hardware-independent ones on any Linux machine or in a container.

CRediT authorship contribution statement

Karol Chlasta: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review and editing. **Grzegorz Marcin Wójcik**: Resources, Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic's Claude to draft and edit the manuscript and to assist with the software and its benchmarking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acknowledgements

The authors thank Maria Curie-Skłodowska University (UMCS) in Lublin and its LubMAN computing centre for access to the "Lunar" cluster [20]; Karol S. Karpowicz and Prof. Przemysław Stpiczyński for setting up and supporting the compute environment; and WarsawIQ for the AMD Radeon 890M on which the OpenCL backend was developed.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System.* Springer, 1998.

[2] Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69–76. doi:10.1016/0020-7101(84)90008-4.

[3] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

[4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[5] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.

[6] Cornelis H, Rodriguez AL, Coop AD, Bower JM. Python as a Federation Tool for GENESIS 3.0. *PLoS ONE* 2012, 7(1):e29018. doi:10.1371/journal.pone.0029018.

[7] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

[8] G.M. Wójcik, W.A. Kamiński. Liquid state machine built of Hodgkin–Huxley neurons and pattern recognition. *Neurocomputing* 58–60 (2004) 245–251.

[9] Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* 1997, 9(6):1179–1209. doi:10.1162/neco.1997.9.6.1179.

[10] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.

[11] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 274–282. doi:10.1109/EMPDP.2019.8671560.

[12] Gewaltig M-O, Diesmann M. NEST (NEural Simulation Tool). *Scholarpedia* 2007, 2(4):1430. doi:10.4249/scholarpedia.1430.

[13] Stimberg M, Brette R, Goodman DFM. Brian 2, an intuitive and efficient neural simulator. *eLife* 2019, 8:e47314. doi:10.7554/eLife.47314.

[14] Yavuz E, Turner J, Nowotny T. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 2016, 6:18854. doi:10.1038/srep18854.

[15] Arbor documentation. Interconnectivity: epochs and the minimal delay. `https://docs.arbor-sim.org/en/stable/concepts/interconnectivity.html`, accessed 17 October 2026.

[16] Vogels TP, Abbott LF. Signal Propagation and Logic Gating in Networks of Integrate-and-Fire Neurons. *Journal of Neuroscience* 2005, 25(46):10786–10795. doi:10.1523/JNEUROSCI.3508-05.2005.

[17] Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, et al. Simulation of networks of spiking neurons: A review of tools and strategies. *Journal of Computational Neuroscience* 2007, 23(3):349–398. doi:10.1007/s10827-007-0038-6.

[18] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.

[19] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.

[20] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. `https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.chtm`, 2015.

Current executable software version

Nr.	(Executable) software metadata description	Please fill in this column
S1	Current software version	v2.6.0
S2	Permanent link to executables of this version	`https://github.com/WarsawIQ/genesis-2.5/releases/tag/v2.6.0`
S3	Legal Software License	GNU General Public License v2 or later
S4	Computing platforms/Operating Systems	Linux x86_64; tested on AlmaLinux 8.10 with CUDA 12.8 (NVIDIA A40, A100) and on Ubuntu 24.04 (TUXEDO OS) with ROCm 6.3.1 and Mesa rusticl 26.1 (AMD Radeon 890M)
S5	Installation requirements & dependencies	glibc and ncurses; for the GPU binaries an NVIDIA driver for CUDA 12, or an OpenCL 1.2 runtime
S6	If available, link to user manual	`README.md` in the repository
S7	Support email for questions	karol@chlasta.pl

Table 5: Software metadata (optional)